THE GITHUB GOLIATHS

TIVIDY

A Generic Workflow Management System

One engine. Two industries.



COS 214 Final Project · 2026

Christian Khaled u25420314 · Keagan Coetzee u25122003 · Nathan Spavins u25054245

Jack Reid u25265718 · Simon Vogel u25087984

github.com/u25420314/COS214_TheGithubGoliaths

Contents

01 Project Overview 3
02 The Team 3
03 Research: Workflow Management Systems 3
04 Development Practices 5
05 System Scope and Definition 6
06 The Two Industries 7
07 Functional Requirements 8
08 Design Patterns 9
09 Activity Diagrams 13
10 Sequence and State Diagrams 16
11 Implementation and Design Decisions 18
12 Testing 19
13 Requirements, Classes and Tests 20
14 Build, Run and Test Instructions 21
15 Repository 21

01 Project Overview

The main focus of this project is that one engine can run two unrelated industries and that no classes have to change for either of the scenario. Adding the insurance process after the admissions process required no change to WorkflowEngine, WorkflowInstance, WorkflowDefinition, WorkflowData, WorkNode, Stage, Task or any Participant. In other words every class in the system apart from the two scenario functions is shared between the two industries exactly as it is written. Everything that differs between the industries sits in one file. Scenario.cpp holds one function per industry and each function describes a process to the director and returns the definition that comes back, along with the people and the roles they hold. No class inside the system mentions a university or an insurer and no engines class knows that more than one scenario even exists.

02 The Team

Each member took responsibility for a distinct part of the design, the implementation and the documentation.

Member	Student no.	Contribution
Nathan Spavins	u25054245	Implemented the Facade, Command and Memento design pattern. As well as help with Integration and doxygen
Keagan Coetzee	u25122003	Implemented the Builder, Composite, Prototype, template Method and the Iterator. As well as help with integration and fixing the pdf for the final submission
Christian Khaled	u25420314	Implement the State and Decorator design pattern. He also fixed the UML and helped with integration and doxygen
Jack Reid	u25265718	Implemented the Chain of Responsibility and Strategy, he helped with creating the different diagrams and helped with doxygen
Simon Vogel	u25087984	Implemented Mediator and Observer, he is the one responsible for the implementation of the TUI

03 Research: Workflow Management Systems

1) What is a workflow and what is a workflow management system?

A workflow is a repeating process where tasks happen in a specific order. A workflow management system is the software that actually runs it, sending each task to the right person when it's their turn.

2) What is the difference between a workflow definition and a running instance?

A definition is the template for a process. An instance is one real case worked through from that template, so one definition can start many separate cases.

3) How are activities or work items represented?

Activities are the steps of a process, and they happen in a set order. Systems often draw them with BPMN (Business Process Model and Notation), a standard set of symbols for drawing business processes. It splits the flow into different paths using decision points called gateways, and parallel paths are tracked separately.

4) How are work items assigned and routed?

The system sends each task to whoever should do it, when it needs doing. Gateways decide which path the work takes next.

5) How are approvals, rejection and escalation handled?

Timers, errors and escalations can be attached to a task as events. Each can stop the task or let it carry on, which is how a slow task gets passed to someone higher.

6) How are participants, roles and responsibilities represented?

Process diagrams use swimlanes to show who does what, and some tasks are marked as work done by people. The system's job is to get each task to the right person.

7) How are dependencies, events and failures handled?

The order of the steps sets the dependencies, so a step waits for the one before it. Messages, timers and errors are events the process can catch, so a failure can be raised and handled.

8) How do workflow systems interact with external systems?

There are tasks for calling other systems and for sending or receiving messages. Automated tasks run alongside human tasks in the same process.

9) How is workflow history or audit information recorded?

The engine stores where each live case is up to, and keeps a separate log of what has already happened.

10) How are running workflows monitored and reported on?

The engine keeps records of current and past cases, and monitoring tools use them to show what is happening. Camunda, a workflow engine, has a web tool called Cockpit for this.

11) What makes a workflow engine reusable across different processes?

A process is written as a model and handed to the engine, which runs it. The engine has no process of its own, so it can run any process. TIVIDY works the same way the engine has no knowledge of any scenario. Both processes are built as WorkflowDefinition objects.

12) Which information belongs to a definition and which to an instance?

The definition holds the structure of the process and its version. The instance holds where that case currently is and what has already happened in it.

13) How do real systems model nested activities or sub-processes?

A process can contain smaller processes, called sub-processes. Each one is tracked as its own part of the running case, and some can be reused from many places. TIVIDY models this with the Composite pattern. A Stage can contain task or further stage objects

14) Which decisions are normally configurable instead of hard-coded?

The process itself is loaded in as a model, not written into the engine. Settings like how much history is kept are also changed through configuration.

15) How are changes to definitions handled while work is in progress?

An updated process is added as a new version next to the old one, so several versions can exist side by side.

References

- Camunda (n.d.) *BPMN 2.0 Implementation Reference*. https://docs.camunda.org/manual/reference/bpmn20/
- Camunda (n.d.) *History and Audit Event Log*. https://docs.camunda.org/manual/7.19/user-guide/process-engine/history
- Camunda (n.d.) *Process Engine Concepts*. https://docs.camunda.org/manual/7.9/user-guide/process-engine/process-engine-concepts
- IBM (n.d.) *What is a workflow?* https://www.ibm.com/topics/workflow

04 Development Practices

4.1 Git and Engineering Practices

Work was divided evenly among the group where each member could implement the design patterns independently, the repository history as well reflects that initial division. Each pattern or component was developed on its own branch so that no member blocked another while a component was incomplete. Completed branches were brought in through pull requests rather than merge directly where over more than 25 pull requests were opened over the time of our project. Merges were carried out by different members and in many if not majority done by someone other than the author of that branch so changes were seen by a second person before they reached shared code. We also had branch protection where at least one other member had to review the persons code before merging directly into dev allowing for code to be double checked before making any drastic changes. Branches taht integrated multiple members work like System-Fixer were used to fix components before they moved further while dev contained the integrated system.

4.2 Continuous Integration

We use GitHub Actions to build and test the project automatically. The settings are in a file called ci.yml. It runs whenever someone pushes a commit to main or dev, and whenever a pull request is opened against either of those branches.

Each run starts on a clean Ubuntu machine, so nothing from anyone's computer is carried over and the project has to build from the repository alone. The run downloads the code, sets it up with CMake using C++11, compiles everything, and then runs the whole doctest suite through CTest. If a test fails, the run prints the test that failed instead of only reporting an error, which makes the problem easy to find. If any of these steps fail, the run is marked as failed and a red mark appears next to the commit or pull request.

05 System Scope and Definition

5.1 What TIVIDY Is Responsible For

TIVIDY is a generic workflow management system. Its responsibilities are:

- **Build and store workflow definitions:** A definition is a reusable plan for a process that is build only once and stored by the facade
- **Create instances from a definition:** Each instance is one real case being worked through, created as a deep copy if the definition so cases never interfere with each other
- **Decide what work comes next:** After every change, the engine rereads the definition tree checks what is finished and works out what tasks are available
- **Send work to people:** Each tasks has a required role stating who is equipped to use handle the task and the engine chooses the participant to give it to.
- **Moves through a progress:** A task can only move through specific transitions, so a task cannot be completed before it has started for example
- **Records History and can undo:** every action is stored as a command with a snapshot of the task before it ran so the most recent action can be reversed
- **It Notifies interested parts of the system:** when something happens every registered listener is told which how an audit log is built
- **It sends tasks through approvers:** a task can carry a chain of approvers that must accept work before it is allowed to complete.

5.2 Boundaries

The boundaries of this system are that it does not integrate with any external systems because the focus is on the workflow design instead of the integration. There is no database and there is no GUI. A state is also held in memory and reached through a facade class TividySystem. The system is also kept separate from the scenarios that run on it. Scenario.cpp contains only code that builds workflow definitions, one function per scenario and main.cpp chooses the scenario, registers it with the facade and drives one case through it. No class inside the system refers to any of the two scenarios. The scenario is data passed into a generic engine.

5.3 Assumptions

We made a few assumptions to keep the project focused on workflow design rather than on infrastructure. The few assumptions we made were that a person is always free to work when a task is given to them. So sick leave or working hours are not apart of the system. And one task is done by one person rather than being shared by many. We also assumed that a persons role is enough to determine if they can do a task so that no other person has permission beyond their role they have. The system has no clock, but the design allows for deadlines and for a task to be escalated after a long wait but nothing specific counts the time while the system runs. SO these are built into the structure rather than something that happens on its own. Everything is also kept in memory while the program runs and nothing is saved afterwards or outside the project.

5.4 Definitions and Instances

The information that is in the workflow definition is the same information for every run, it stores the workflow name, the stages, tasks, order that they happen, parallel tasks, roles needed, rules, and approvals. The instance rather is a working copy of the definition. Each instance has its own specific data and changing it doesn't change others or the definition, just its own data. It's like an object class (definition), and the actual objects (instances).

5.5 What Must Vary Between Organisations

Tividy needs to run for organisations where processes have nothing in common, so anything that differs between them is kept out of the engine. The process changes completely and is described as a tree of stages and tasks rather than written directly into the actual system. Ordering differs as well, so each stage is marked as either parallel or sequential. The document checks and the three assessment tasks run at the same time, while the steps run in order. Roles and participants are plain data that are supplied when you setup the system. Admissions needs an Admissions Officer and a Head of Degree while a claim needs a Claims Clerk, an Assessor and a Claims Manager.

How work is assigned changes depending on one organisation gives a task to anyone with the required role and another to whoever is least busy, so the rule is a strategy that can be swapped. Approval varies, as some tasks need one approver, some need several in sequence and most need none, so approvers form a chain attached to a task. Extra behaviour on a task such as validation, a deadline or a priority is wrapped around the task rather than built into it, so a task can have any combination without a new class for each. Finally, organisations reacts differently, so listeners are registered with the engine and told what happened without the engine knowing what they do.

06 The Two Industries

6.1 Scenario 1 — Goliath University (Education)

- **Organisation:** a university receiving many undergraduate applications, where tracking each application's stage and owner is slow to do by hand.
- **Process managed:** one student application, from submission to the applicant receiving a decision.
- **Participants and roles:** Admissions Officer (checks documents and payment), Head of Degree (reviews), Head of Faculty (higher approval above the Head of Degree).
- **Major work:** check eligibility, verify ID, confirm payment, review the application, send the decision.
- **Decisions:** are the documents valid, has payment cleared, does the Head of Degree accept the application.
- **Dependencies:** the review cannot start until both document checks are done, and the decision cannot be sent until the review is complete.
- **Failures:** missing documents, failed payment, no response from a reviewer, applicant cancels.

- **External systems:** a payment system and an email system would be used in practice, but are out of scope here.
- **Outcome:** a final decision reaches the applicant, with every action recorded.

6.2 Scenario 2 — Goliath Insurance (Insurance)

- **Organisation:** an insurer receiving motor claims, where every payout must be approved before money leaves the business.
- **Process managed:** one claim for hail damage to a vehicle, from capture to closure.
- **Participants and roles:** Claims Clerk (captures and verifies), Assessor (inspects damage), Fraud Investigator (runs the fraud check), Claims Manager (approves the payout), Finance Officer (pays the claimant).
- **Major work:** capture the claim, verify the policy, assess damage, run a fraud check, check cover, approve the payout, pay, close.
- **Decisions:** is the policy active and does it cover the incident, does the fraud check pass, is the damage assessment accepted, does the Claims Manager approve the payout.
- **Dependencies:** no assessment may begin until intake is complete, and settlement may not begin until all three assessment tasks are complete, so a rejected assessment holds up the whole claim.
- **Approval rule:** "Approve payout" carries a RoleApprover requiring the Claims Manager role, so no one else can complete it.
- **Failures:** a lapsed policy, a failed fraud check, an incomplete damage report that must be reassigned to another assessor, an action performed by mistake that must be undone.
- **Outcome:** the claim is paid and closed or declined, with the full sequence of events in the audit log.

07 Functional Requirements

Each requirement is paired with the design pattern that satisfies it and the reasoning behind that choice.

#	Functional requirement	Design pattern	Reasoning behind the choice
FR 1	The system builds a workflow definition from a step-by-step description of stages and tasks without the caller assembling the tree itself.	**Builder**	Separates how a process is described from how it is assembled, so a new process is written as instructions rather than as tree-building code.
FR 2	The system shall represent a process as a tree in which a stage may contain tasks or further stages, and treat both uniformly.	**Composite**	Allows nested sub-processes such as Document Checks and Assessment to be handled with the same code as a single task.
FR 3	The system shall traverse a definition in a defined order without exposing how it is stored.	**Iterator**	Lets the engine and the tests walk the tree without depending on the container used inside a stage.

#	Functional requirement	Design pattern	Reasoning behind the choice
FR 4	The system shall start independent instances of a definition, each with its own tasks, state and data.	**Prototype**	Deep-copying the tree guarantees that progress in one case cannot affect another, which the specification requires.
FR 5	The system shall coordinate progression centrally so that tasks never refer to one another.	**Mediator**	Keeps the dependency logic in one place and prevents a web of references between tasks.
FR 6	The system shall determine which tasks become available after each change, honouring sequential and parallel ordering.	**Composite with Mediator**	A stage answers for its own children, so the engine never inspects node types to decide what is ready.
FR 7	The system shall assign a task to a participant holding the required role, by a rule that can be changed per organisation.	**Strategy**	Allows role-based, least-loaded and rule-based assignment to be interchanged without altering the task.
FR 8	The system shall enforce a task lifecycle and refuse any illegal transition.	**State**	Each state answers for the actions legal from it, which removes conditional checks spread through the task.
FR 9	The system shall require approval on nominated tasks, through one or more approvers in sequence.	**Chain of Responsibility**	Supports organisations needing no approver, one approver or several, without changing the task.
FR 10	The system shall record every action as an object so that the most recent action can be undone.	**Command**	Encapsulating actions gives a uniform history and an undo facility that the facade can expose.
FR 11	The system shall capture a task's internal state before it is changed, so the change can be reversed.	**Memento**	Lets a command restore a task without knowing what a task contains.
FR 12	The system shall publish each event to every registered listener.	**Observer**	New reactions such as auditing, notification and progress tracking can be added without the engine knowing what they do.
FR 13	The system shall expose a single entry point through which a client starts and drives a workflow.	**Facade**	Hides the engine, the command history and the definitions behind one class.
FR 14	The system shall allow extra behaviour such as validation, a deadline or a priority to be added to a task.	**Decorator**	Avoids a separate task class for every combination of optional behaviour.
FR 15	The system shall apply a fixed order of operations to every unit of work.	**Template Method**	Guarantees that the guard, the work and the completion step always run in the same order.

08 Design Patterns

Fourteen Gang of Four patterns are used in the system. Each entry gives what the pattern does in TIVIDY, followed by the classes that fill each role.

Builder

Describes a process step by step and assembles it into a definition, so the same build instructions can create a workflow without the caller even touching the tree

- **Builder:** `WorkflowBuilder`
- **ConcreteBuilder:** `StandardWorkflowBuilder`
- **Director:** `WorkflowDirector`
- **Product:** `WorkflowDefinition`
- **Client:** `Scenario.cpp`

Chain of Responsibility

Lets a task be approved by a sequence of approvers, where each one either decides or passes the request to the next.

- **Handler:** `ApprovalHandler`
- **ConcreteHandler:** `RoleApprover, TimeOutEscalationHandler`
- **Client:** `Task`

Command

Turns each action into an object so it can be executed, stored and undone

- **Command:** `Command`
- **ConcreteCommands:** `StartWorkFlowCommand, AssignTaskCommand, StartTaskCommand, CompleteTaskCommand, RejectTaskCommand`
- **Invoker:** `CommandHistory`
- **Receiver:** `WorkflowEngine`
- **Client:** `TividySystem`

Composite

Stores the process as a tree so a stage and a single task can be treated the same way and a stage can also contain stages.

- **Component:** `WorkNode`
- **Composite:** `Stage`
- **Leaf:** `Task`
- **Client:** `WorkflowEngine`

Decorator

wraps extra behaviour around a task without changing the task class, so a task can be validated, given a deadline or given a priority in any order.

- **Component:** `WorkNode`
- **ConcreteComponent:** `Task`
- **Decorator:** `TaskDecorator`
- **ConcreteDecorator:** `ValidationDecorator, DeadlineDecorator, PriorityDecorator`

Facade

Brings the whole system behind one single class so that a client starts and can drive a workflow without knowing the the system

- **Facade:** `TividySystem`
- **Subsystem classes:** `WorkflowEngine, CommandHistory, WorkflowDefinition, AuditLogger`

Iterator

Walks the composite tree without exposing how it is stored

- **Iterator:** `WorkNodeIterator`
- **ConcreteIterator:** `DepthFirstIterator`
- **Aggregate:** `WorkNode`
- **ConcreteAggregate:** `Stage`

Mediator

keeps tasks from knowing about each other. A task only reports what happens to the engine and then decides what to do next and who it specifically goes to

- **Mediator:** `WorkFlowMediator`
- **ConcreteMediator:** `WorkFlowEngine`
- **Collegue:** `WorkNode`
- **ConcreteCollegue:** `Task, Stage`

Memento

Saves a task's internal state before a command changes it, so the action can be reversed without the command knowing what a task contains.

- **Originator:** `Task`
- **Memento:** `TaskMemento`
- **Caretakers:** `AssignTaskCommand, StartTaskCommand, CompleteTaskCommand, RejectTaskCommand`

Observer

Publishes each event to every registered listener, so new reactions can be added without the engine knowing what specifically they do

- **Subject:** `WorkflowSubject`
- **ConcreteSubject:** `WorkflowEngine`
- **Observer:** `WorkFlowObserver`
- **ConcreteObservers:** `AuditLogger, NotificationService, ProgressMonitor`

State

Holds a task's current state and decides which actions are legal from it. A task moves Available -> Assigned -> inProgress -> Completed with Rejected, Escalated and Cancelled as other outcomes. Any illegal transition is refused

- **Context:** `Task`
- **State:** `TaskState`
- **ConcreteStates:** `AvailableState, AssignedState, inProgressState, CompletedState, RejectedState, EscalatedState, CancelledState`

Strategy

Lets the rule for choosing who gets a task be swapped without changing tha task

- **Context:** `Task`
- **Strategy:** `AssignmentStrategy`
- **ConcreteStrategies:** `RoleBasedAssignment, LeastLoadedAssignment, RuleBasedAssignment`

Template Method

WorkNode::run() fixes the order of every unit of work: canStart(), calls doWork(), then finish().

- **AbstractClass:** `WorkNode`
- **ConcreteClasses:** `Task, Stage`

Prototype

Deep-copies a definition's tree so each running case has its own tasks with their own state, and completing a task in one case cannot affect another.

- **Prototype:** `WorkNode`
- **ConcretePrototypes:** `Task, Stage TaskDecorator`
- **Client:** `WorkflowDefinition`

09 Activity Diagrams

Diagram 1 — Generic Workflow Execution

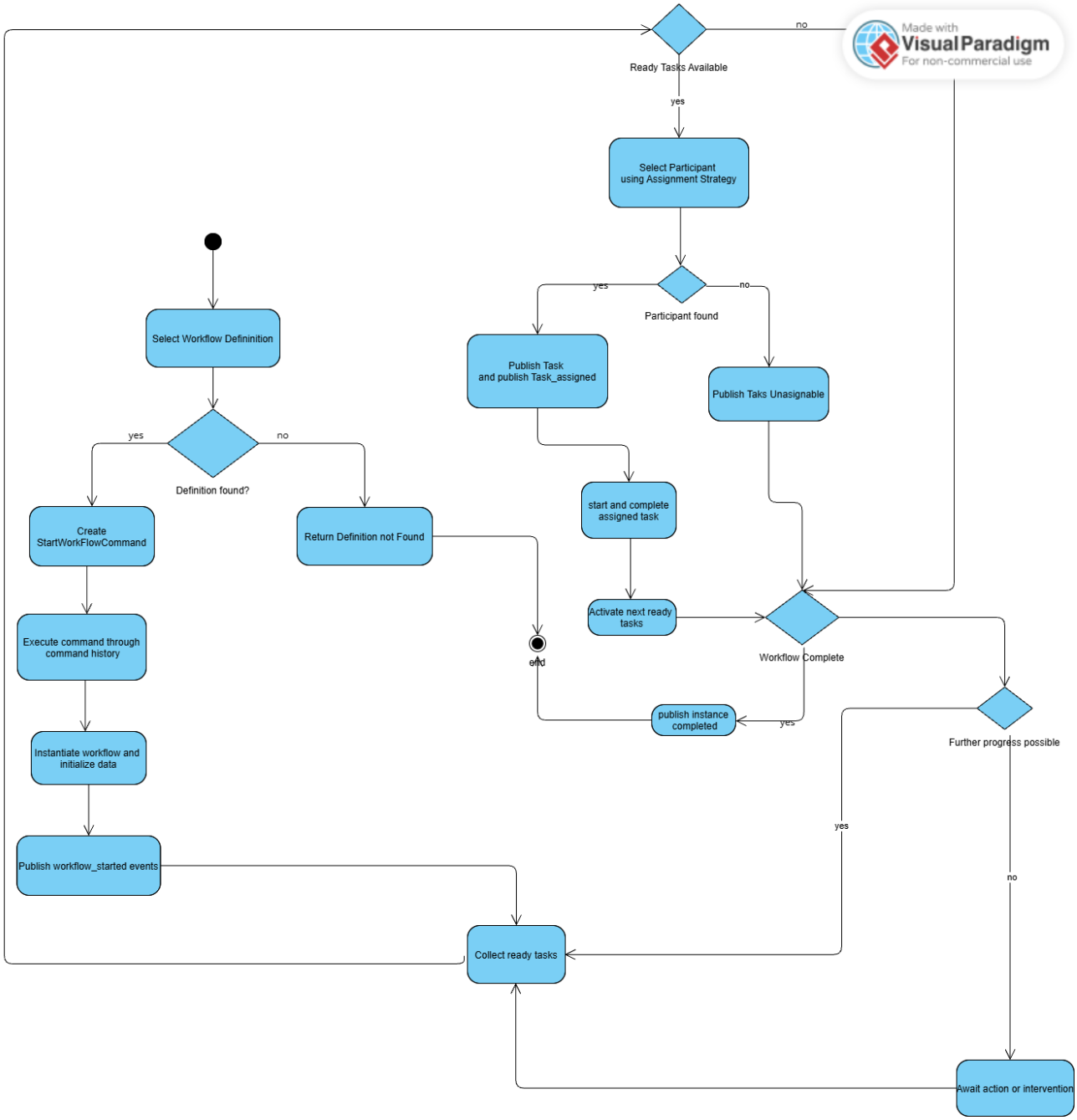


Diagram 1: Generic Workflow Execution

Diagram 2 — Generic TIVIDY Workflow

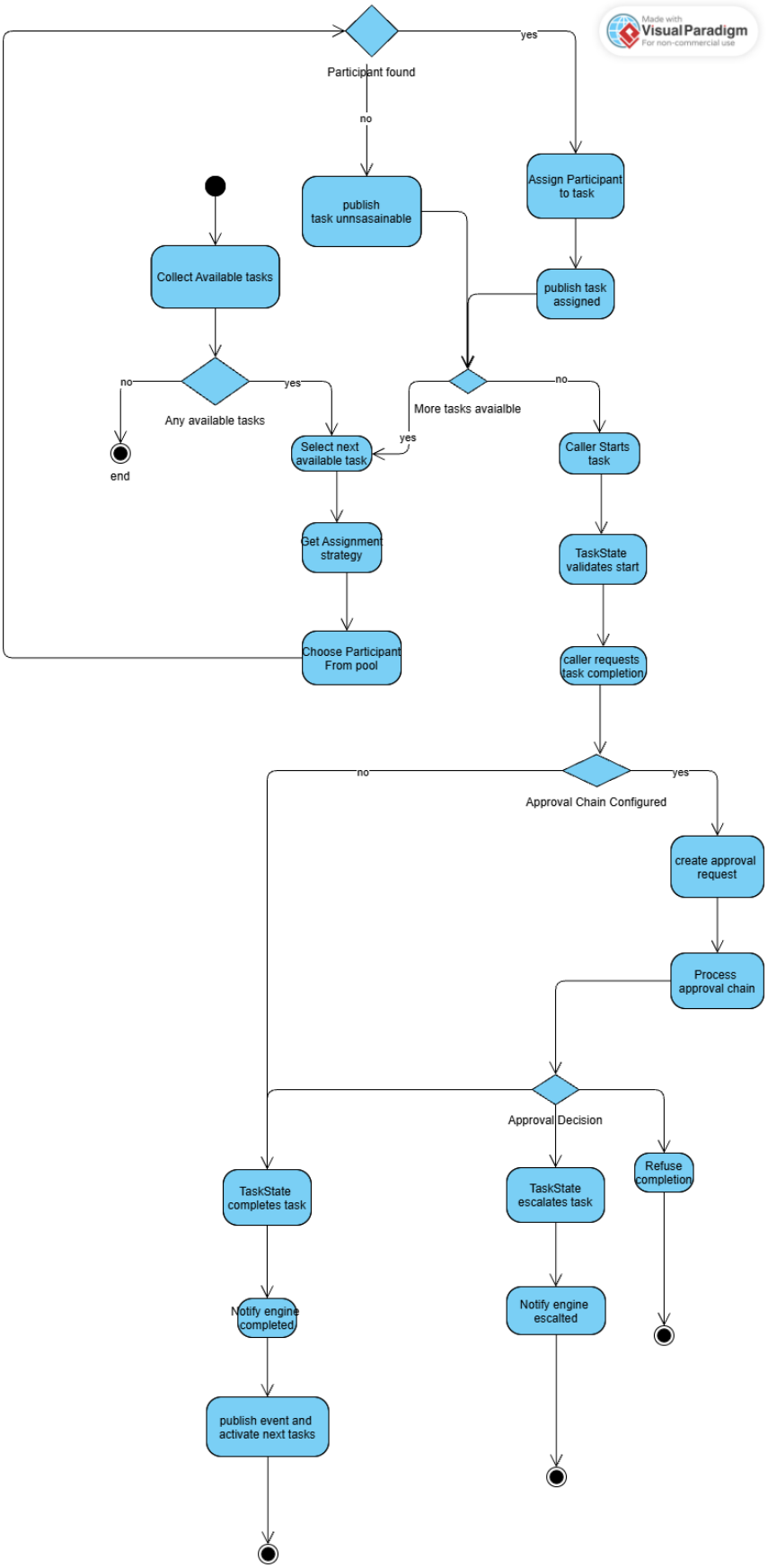


Diagram 2: Generic TIVIDY workflow

Diagram 3 — Scenario Based Workflow Diagram

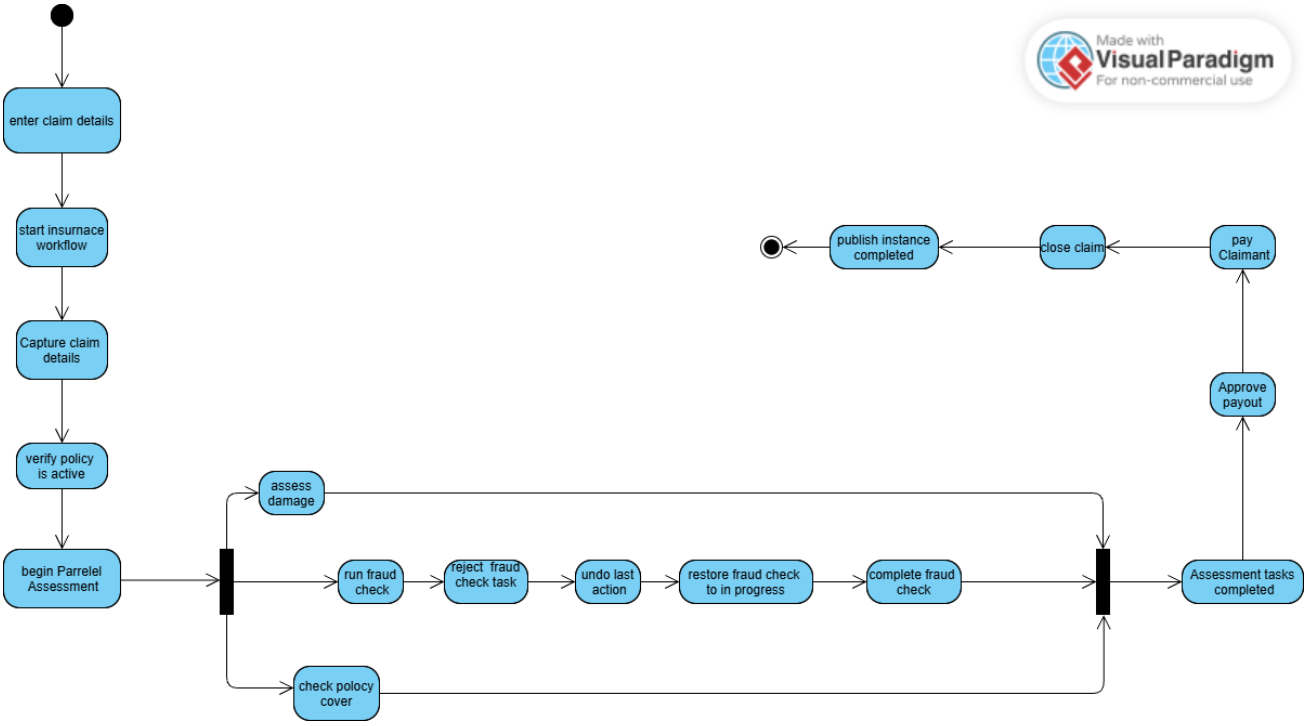
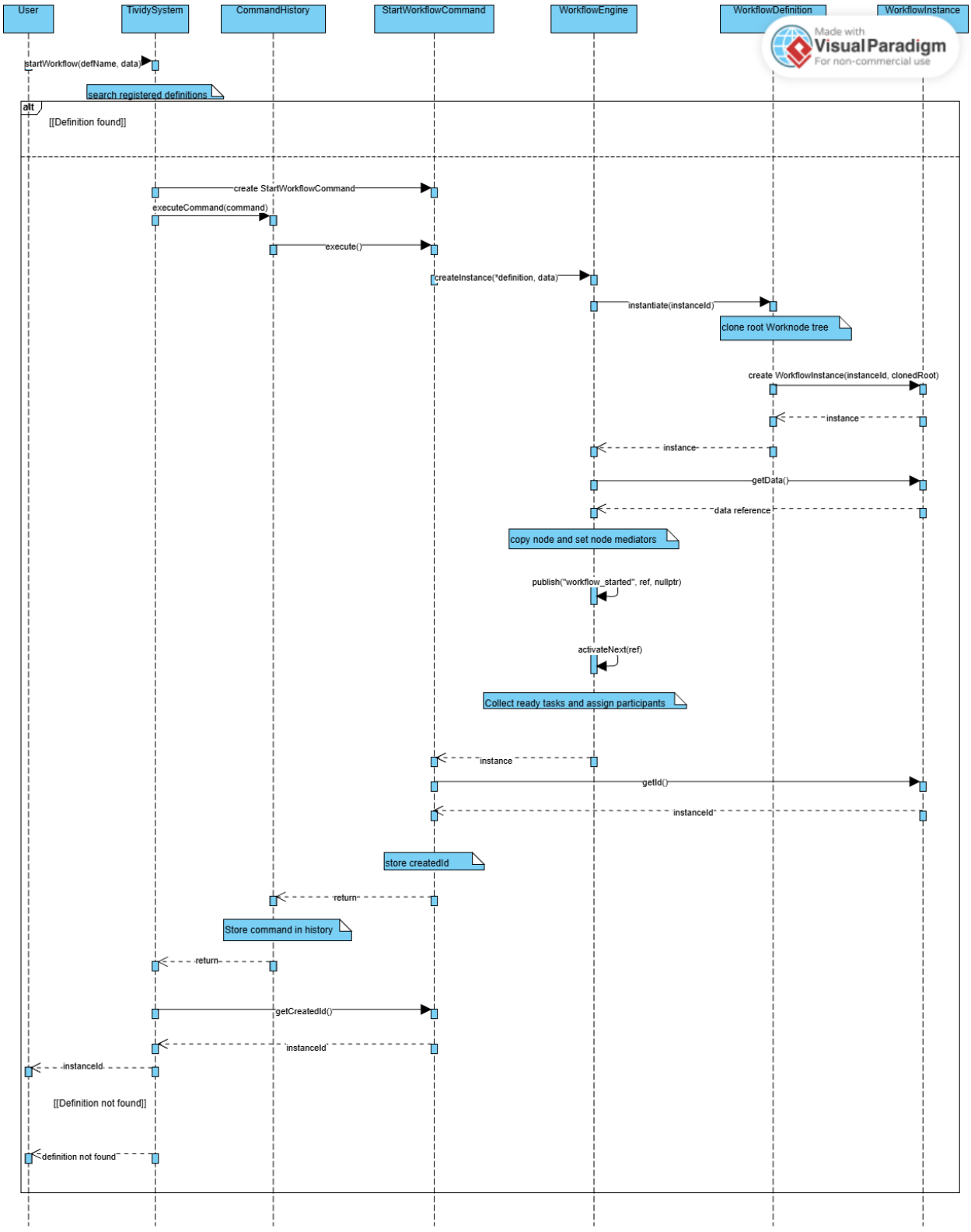


Diagram 3: Scenario based workflow diagram — Goliath Insurance claim

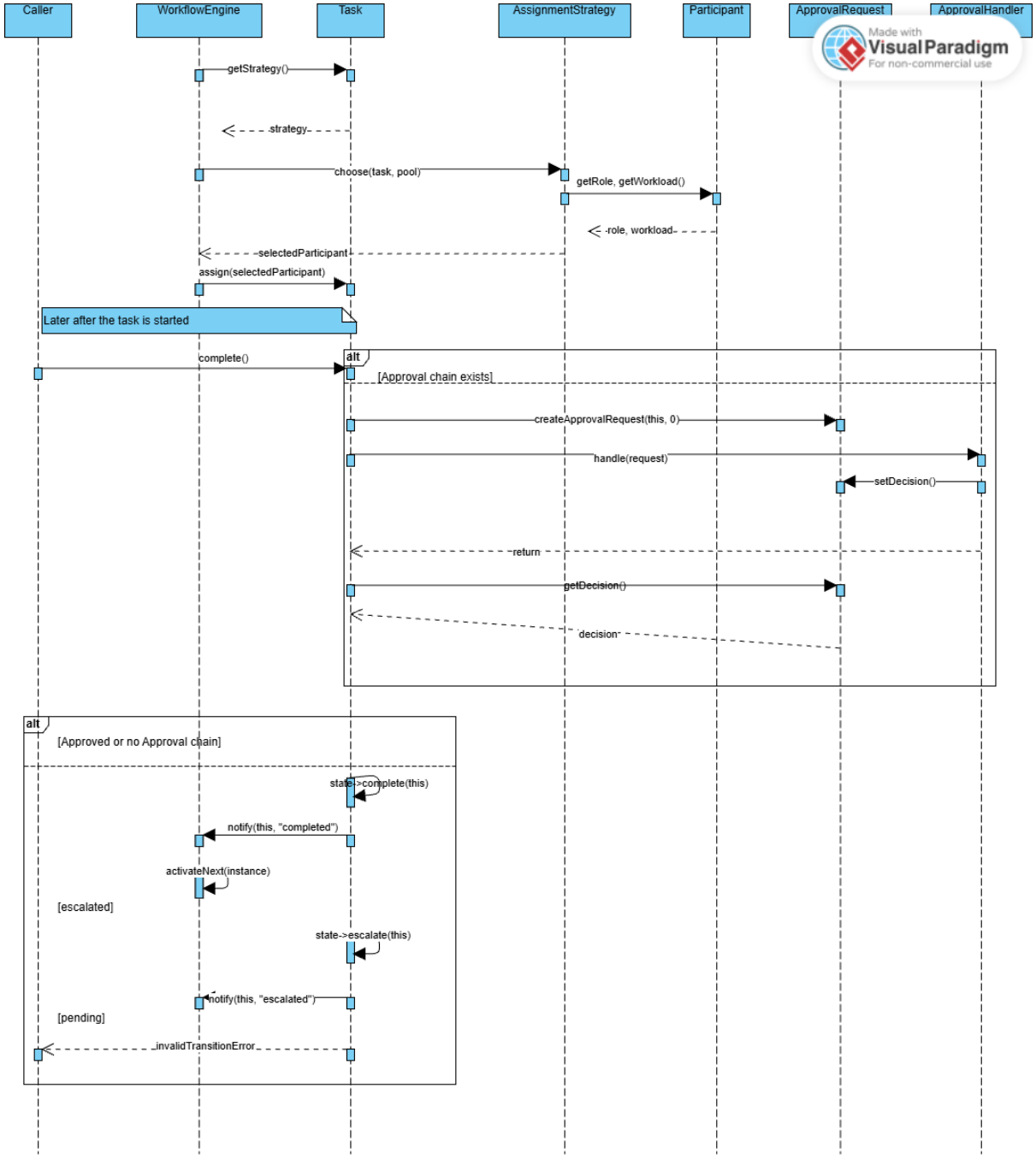
10 Sequence and State Diagrams

Sequence Diagram 1 — Starting a Workflow Instance



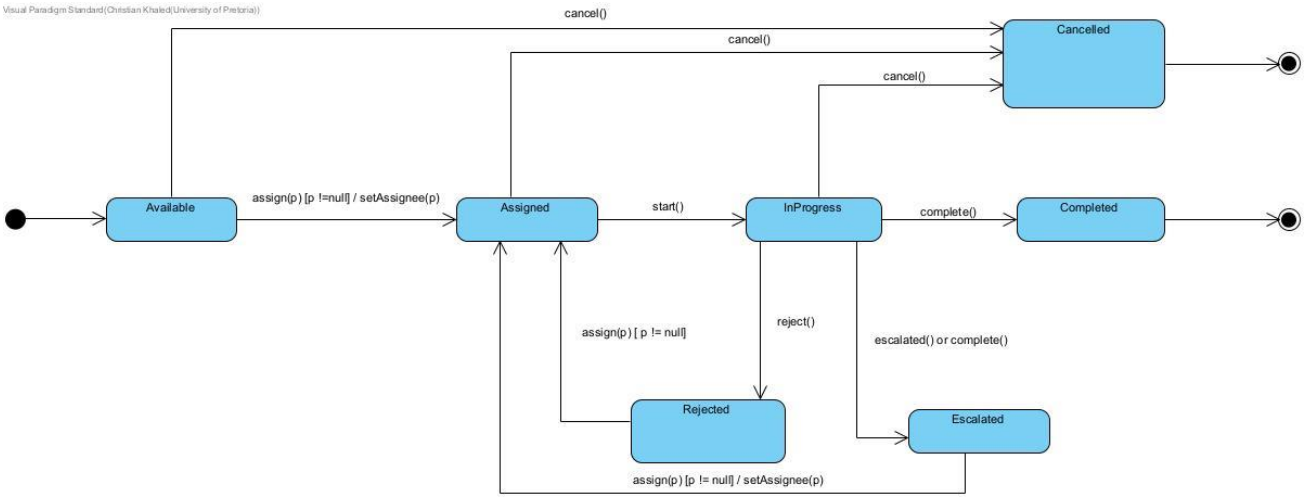
Sequence Diagram 1

Sequence Diagram 2 — Assigning and Completing a Task



Sequence Diagram 2

State Diagram



Task lifecycle

11 Implementation and Design Decisions

TIVIDY is implemented in C++11 and built with CMake. Every source file other than the tests is compiled into a static library, tividy_core, which both the application and the test runner link against, so the behaviour the tests exercise is the same behaviour a marker sees at run time. The project can also be built directly with g++ for anyone without CMake, and the doctest framework is included in the repository as a single header, so nothing needs installing before the system can be built or tested.

The first and most consequential decision was to separate the plan from the run.

A WorkflowDefinition is built once and never changes while cases are being processed, and starting a case deep-copies that definition into a WorkflowInstance with its own tasks and its own state. This follows the definition and instance distinction described in our research, and it guarantees that completing a task in one case cannot affect another.

The second decision was to keep type checking out of the engine.

An earlier version worked out which tasks were ready by walking the tree and using dynamic_cast to tell a stage from a task, which put knowledge of the tree's shape inside the engine rather than inside the tree itself. This was replaced by a collectReady operation declared on WorkNode, where a stage answers by consulting its own children depending on whether it is sequential or parallel, and a task answers for itself. The engine now asks the root one question and receives the list of tasks that may be worked on.

The third decision was to complete the set of commands.

Our original design had only three concrete commands, which started a workflow, assigned a task and completed a task. Once the task lifecycle was implemented, this left two of its transitions with no command behind them: a task could be started and rejected, but neither action went through the command history, so neither could be undone and neither appeared in the record of what had happened. We added StartTaskCommand and RejectTaskCommand so that every action a participant can take is a command.

The benefit is uniformity. Undo works the same way for any action, and the history is a complete account of the case rather than a partial one.

The fourth decision concerned ordering.

We had originally planned separate subclasses for sequential and parallel stages, but the two differ only in which children they expose as ready and in nothing else at all. Everything else, how children are stored, how completion is judged, how the mediator is passed down, would have been identical in both. Rather than duplicate a whole class for one difference, a stage carries a flag saying which kind it is, and the one place that behaviour differs reads that flag. A sequential stage offers only its first unfinished child, while a parallel stage offers all of them.

The fifth decision was to change how approval works.

We first planned for tasks to move along a chain of approvers based on how long they had been waiting, so that a task nobody acted on would escalate upward on its own. This turned out to be far more complicated than it was worth, because it needs the system to track time while it runs, and that would have pulled in a whole mechanism unrelated to workflow structure. We changed it so that a task is handled according to the role it requires instead. A RoleApprover checks that whoever is completing the task holds the right role, which is what prevents the payout in the insurance process being approved by anyone other than a Claims Manager. The timeout handler remains part of the design and shows how the chain extends, but the approval actually used in both industries is role-based.

Smaller corrections

Three smaller corrections were made as the implementation progressed. The Adapter pattern was removed once the system boundary was fixed at no external integration, since its stubs represented services that were never called. A duplicate AvailableState class created under two spellings was reduced to one. The builder was corrected after unit testing revealed that addTask did not record the task it had added as the last node, which meant a decorator applied straight afterwards attached to the wrong node.

12 Testing

We used automated testing uses doctest, a single-header framework included in the repository. We chose it over GoogleTest because it needs no installation and its interface is small enough that every member could write tests without first learning a new framework.

The tests live in a “tests” directory with their own CMake configuration, which builds a tividy_tests executable, links it against tividy_core and registers it with CTest. The tests therefore use exactly the same library the application uses.

The suite contains a total of 104 test cases across 17 files. The specification names four behaviours as important and each is covered. Transitions and invalid operations are tested in test_task_states.cpp where it checks every legal move through the lifecycle and confirms that illegal ones like completing a task that was never started, are refused. Routing is tested through the three assignment strategy files and the coordination tests. Instance isolation is tested in test_composite_behaviour.cpp, which checks that cloning

a definition produces a genuinely deep copy with fresh task state. Every member of the group contributed tests as well as implementation.

13 Requirements, Classes and Tests

Every functional requirement can be followed through to the classes that implement it and the test that proves it works. The most important of these are the ones the specification singles out. FR 4, that each instance runs independently, is implemented by the clone operation on WorkNode with Stage and Task, and is proven by test_composite_behaviour.cpp, which checks that a cloned task carries fresh state rather than the state of the original. FR 8, the task lifecycle, is implemented by TaskState and its seven concrete states and is proven by test_task_states.cpp, which checks both the legal transitions and the refusal of illegal ones. FR 7, routing work to the right person, is implemented by AssignmentStrategy and its three implementations and is proven by a test file for each. The table below gives the same chain for every requirement, so any claim made in this report can be checked against the code and the suite.

#	Implemented by	Verified by
FR 1	`WorkflowDirector`, `StandardWorkflowBuilder`, `WorkflowDefinition`	`test_builder.cpp`
FR 2	`WorkNode`, `Stage`, `Task`	`test_composite_structure.cpp`
FR 3	`WorkNodeIterator`, `DepthFirstIterator`	`test_composite_structure.cpp`
FR 4	`WorkNode::clone`, `Stage`, `Task`	`test_composite_behaviour.cpp`
FR 5	`WorkflowEngine`, `WorkflowMediator`	`test_coordination.cpp`, `test_workflow_engine.cpp`
FR 6	`Stage::getReadyChildren`, `WorkNode::collectReady`	`test_composite_behaviour.cpp`
FR 7	`AssignmentStrategy` and its three implementations	`RoleBasedAssignment_test.cpp`, `LeastLoadedAssignment_test.cpp`, `RuleBasedAssignment_test.cpp`
FR 8	`TaskState` and the seven concrete states	`test_task_states.cpp`
FR 9	`ApprovalHandler`, `RoleApprover`, `TimeoutEscalationHandler`, `ApprovalRequest`	`RoleApprover_test.cpp`, `TimeoutEscalationHandler_test.cpp`, `ApprovalRequest_test.cpp`
FR 10	`Command` and the five concrete commands, `CommandHistory`	`test_task_commands.cpp`, `test_command_history.cpp`
FR 11	`TaskMemento`, `Task::createMemento`, `Task::restore`	`test_command_history.cpp`
FR 12	`WorkflowObserver`, `AuditLogger`, `NotificationService`, `ProgressMonitor`	`test_coordination.cpp`, `test_facade.cpp`
FR 13	`TividySystem`	`test_facade.cpp`
FR 14	`TaskDecorator`, `ValidationDecorator`, `DeadlineDecorator`, `PriorityDecorator`	`test_decorators.cpp`
FR 15	`WorkNode::run`	exercised by every behavioural test

14 Build, Run and Test Instructions

The project requires a C++11 compiler and CMake 3.10 or later. Doxygen is optional and is needed only to regenerate the code documentation. Nothing else has to be installed, because the doctest framework used by the test suite is included in the repository as a single header at tests/doctest.h.

Building and running with CMake

```
cmake -S . -B build
cmake --build build
./build/tividy
```

Building and running without CMake

```
g++ -std=c++11 -I. *.cpp -o tividy
./tividy
```

Running the tests with CMake

```
cmake -S . -B build
cmake --build build
ctest --test-dir build --output-on-failure
```

Running the tests without CMake

```
g++ -std=c++11 -I. -Itests $(ls *.cpp | grep -v "^main.cpp$") tests/*.cpp -o
tividy_tests
./tividy_tests
```

The --output-on-failure flag makes CTest print the output of any failing test rather than only reporting that the run failed, which is how the continuous integration pipeline runs the suite as well. Running tividy_tests directly produces the same result but lists each test case as it passes, which is more useful while working on a single area of the system.

15 Repository

Github Link

https://github.com/u25420314/COS214_TheGithubGoliaths



The Github Goliaths · COS 214 · 2026